

多模态 Agent Post-Training 项目

完整架构文档

2026 年 5 月 14 日

摘要

本文档对多模态 Agent Post-Training 项目的全部代码文件、数据文件、模型权重进行系统性梳理。对每个文件说明其功能定位、解决的问题、在整体流程中的位置，以及文件之间的协作关系。文档按模块组织—Agent 推理核心 (agent/)、训练引擎 (train/)、评测系统 (eval/)、数据构建管线 (scripts/)、训练数据 (data/)、模型权重 (LoRA adapters)、外部图片资源—覆盖从数据准备到模型部署的完整链路。

目录

1	项目总览	1
1.1	核心问题	1
1.2	整体架构	1
2	Agent 推理核心 (agent/)	2
2.1	runtime.py — 多轮 Agent 执行引擎	2
2.2	tools.py — 工具实现	2
2.3	vlm.py — VLM 模型封装	3
2.4	parser.py — 模型输出解析	3
2.5	prompts.py — 提示词模块	3
3	训练引擎 (train/)	3
3.1	train_sft_lora.py — SFT LoRA 训练	3
3.2	train_dpo_lora.py — DPO 偏好训练	4
4	评测系统 (eval/)	4
4.1	run_vlm_eval.py — VLM 评测主脚本	4
4.2	run_seed_eval.py — 早期评测	4
5	数据构建管线 (scripts/)	5
5.1	核心数据构建	5
5.1.1	build_first_tool_sft.py	5
5.1.2	build_second_tool_sft.py	5

目录	2
5.1.3 build_sft_train.py	5
5.1.4 build_agent_train_v3.py	5
5.1.5 build_agent_train_real_v3.py	5
5.2 外部数据集转换	5
5.2.1 convert_chartqa_to_sft.py	5
5.2.2 convert_docvqa_to_sft.py	6
5.2.3 convert_cord_to_sft.py	6
5.3 真实 VLM 数据重建	6
5.3.1 real_vision_parse.py	6
5.3.2 rebuild_seed_with_real_vlm.py	6
5.4 数据增强与修正	6
5.4.1 build_augmented_v5_data.py	6
5.4.2 build_hard_cases_from_eval.py	6
5.4.3 build_dpo_multiturn_v4.py	6
5.4.4 build_refusal_samples.py	6
5.4.5 build_tool_decision_samples.py	7
5.5 诊断与分析	7
5.5.1 tag_vlm_errors.py	7
5.5.2 analyze_sft_train.py, analyze_vlm_eval.py, analyze_error_tags.py	7
5.5.3 add_eval_category.py, check_seed_data.py	7
6 训练数据 (data/)	7
6.1 SFT 训练数据	7
6.2 评测数据	7
6.3 DPO 偏好数据	8
6.4 规则文档 (data/docs/)	8
6.5 图片资源 (data/images/)	8
7 模型权重 (autodl-tmp/multimodal-agent-lora/)	8
8 外部图片资源 (autodl-tmp/multimodal-agent-data/)	9
9 完整 workflow	9
9.1 训练数据构建流程	9
9.2 训练流程	9
9.3 评测流程	9
9.4 推理流程	10
10 文件协作关系图	10
11 总结	11

1 项目总览

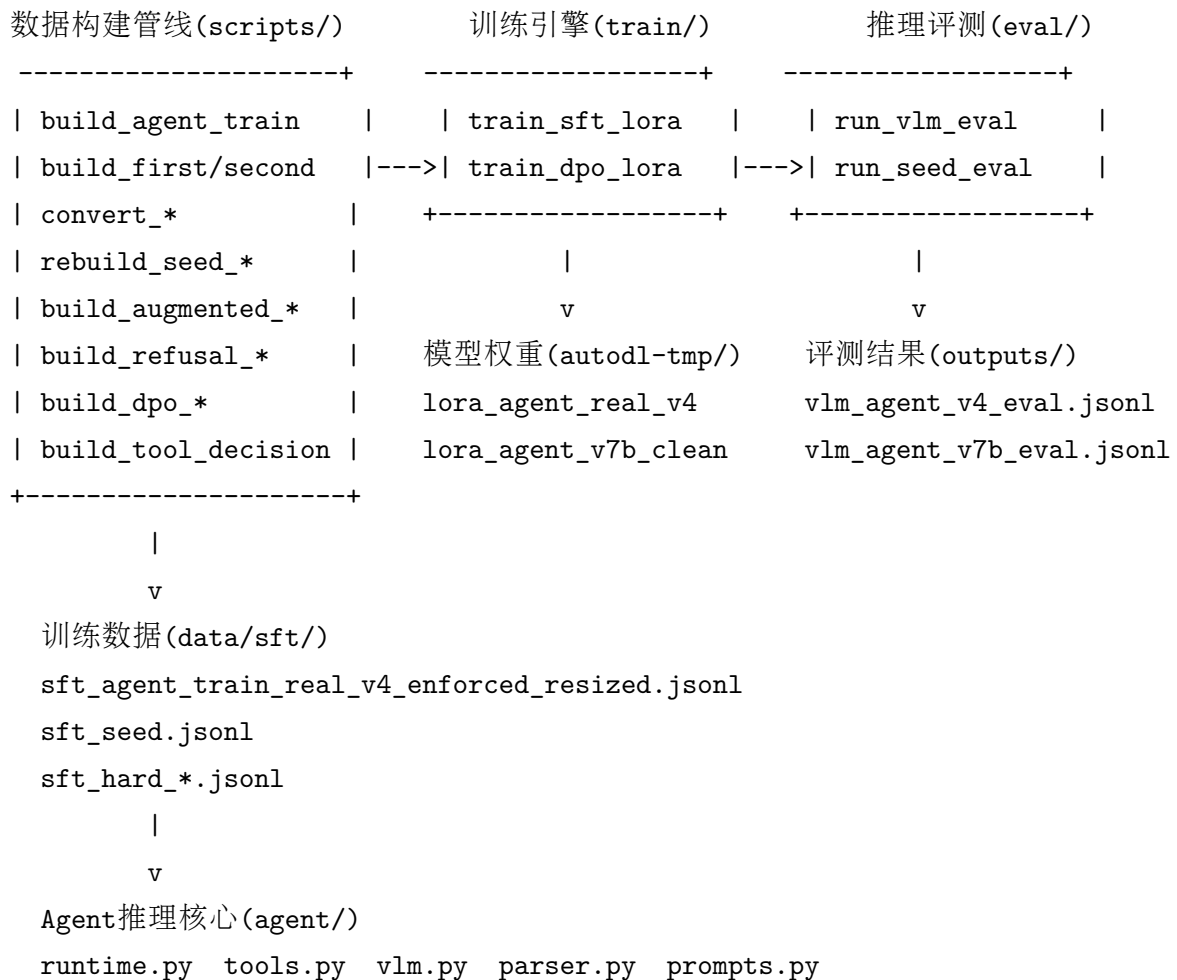
1.1 核心问题

给定一张图片和一个自然语言问题，训练 VLM 严格按照工具调用流程完成任务：

```
User(图片+问题) -> vision_parse(OCR/图表/描述)
-> retrieve_docs(查规则) / python_exec(计算)
-> final_answer(给出依据) 或 合理拒答
```

核心约束：每轮对话模型只能输出一个 `<tool_call>` 或 `<final_answer>`，不得输出裸文本或多个动作。

1.2 整体架构



2 Agent 推理核心 (agent/)

2.1 runtime.py — 多轮 Agent 执行引擎

解决的问题：管理 VLM 的多轮工具调用循环。接收用户消息，让 VLM 生成响应，解析 `tool_call` 或 `final_answer`，执行工具，将工具结果反馈给模型，循环直到 `final_answer` 或达到 `max_steps`。

核心逻辑：

1. 加载 system prompt + 用户消息 (图片 + 问题) -> messages
2. 调用 VLM 生成 -> 解析输出 (parser.parse_model_output)
3. 如果是 tool_call -> 执行对应工具 -> 工具结果 append 到 messages -> 回到步骤 2
4. 如果是 final_answer -> 检查 enforce_required_tools
5. 如果漏调必要工具 -> 拦截答案, 注入纠正提示 -> 回到步骤 2
6. 如果通过 -> 返回 answer + trace

协作关系: 依赖 tools.py (工具执行)、parser.py (输出解析)、prompts.py (系统提示词 + 纠正提示词模块)、vlm.py (模型推理)。

Enforce 机制: 通过关键词匹配判断问题类型, 如果问题是发票类 + 未调 retrieve_docs, 或图表类 + 未调 python_exec, 拦截 final_answer 并注入纠正提示。这是 V2 从 50% tool hit 提升到 100% 的关键。

2.2 tools.py — 工具实现

解决的问题: 提供 vision_parse (视觉解析)、retrieve_docs (规则检索)、python_exec (数学计算) 三个工具的真实实现。

vision_parse:

- 支持 4 种模式: ocr、caption、chart_values、ocr+caption
- 真实 VLM 推理: 当 _vlm_model 被注入时, 调用 Qwen-VL 进行视觉推理
- Mock 回退: 当 VLM 不可用时, 根据文件名模式返回预设模拟结果
- 画像缓存: 对外部数据集使用 image+mode 级缓存避免重复推理

retrieve_docs:

- 加载 data/docs/下的规则文档 (invoice_rules.txt, chart_rules.txt, product_rules.txt)
- 通过关键词-> 文件映射检索对应规则
- 支持中英文关键词匹配

python_exec:

- 正则白名单安全验证: 只允许数字、运算符、空格和括号
- 在空 builtins 沙箱中 eval 执行

协作关系: 被 runtime.py 调用。vision_parse 依赖 vlm.py 的 QwenVLModel 通过 set_vlm_model() 注入。

2.3 vlm.py — VLM 模型封装

解决的问题: 封装 Qwen2.5-VL 模型的加载、推理、LoRA adapter 注入、以及 mock 模式回退。

核心类 QwenVLModel:

- __init__: 加载模型和 processor, 支持 adapter 注入和 mock 回退
- generate: 接收 messages 列表, 返回模型生成的文本
- mock_generate: 当 VLM 不可用时, 根据用户文本和图片名推断应输出什么 (用于跑通流程)

协作关系: 被 runtime.py (Agent 推理)、eval/run_vlm_eval.py (评测) 使用。当 adapter_name 参数非空时, 通过 PeftModel 加载 LoRA 权重。

2.4 parser.py — 模型输出解析

解决的问题：将 VLM 的原始文本输出解析为结构化的 `tool_call` 或 `final_answer`。

核心函数 `parse_model_output`:

1. 正则匹配 `<tool_call>...</tool_call>` -> 提取 JSON -> 返回 `{type: tool_call, name, args}`
2. 容错：如果缺闭合标签，尝试提取 JSON 对象
3. 正则匹配 `<final_answer>...</final_answer>` -> 返回 `{type: final_answer, content}`
4. 容错：如果缺闭合标签但内容非空，尝试恢复
5. 以上都不匹配 -> 返回 `{type: parse_error}`

协作关系：被 `runtime.py` 和 `eval` 系统调用。是格式纪律的最后一道防线。

2.5 prompts.py — 提示词模块

解决的问题：管理系统提示词模板。定义 VLM 的工具调用格式要求（每轮只能一个 `tool_call` 或 `final_answer`）。

3 训练引擎 (train/)

3.1 train_sft_lora.py — SFT LoRA 训练

解决的问题：对 Qwen2.5-VL 基座模型进行 LoRA 微调，使模型学会多轮工具调用流程。

核心机制：

- `AgentSFTDataset`: 读取 JSONL 训练数据，通过 `expand_assistant_turns` 将每条多轮对话拆分为多个训练样本（每个 `assistant turn` 作为一个样本）
- `QwenVLSFTCollator`: 对每个样本，计算 `prefix`（目标 `turn` 之前的所有内容）和 `full`（包含目标 `turn`），在 `labels` 中将 `prefix` 位置的 token 设为 -100（不计算 loss），只对 `assistant` 输出的 token 计算 loss
- LoRA 配置: `r=16/32/64`, `alpha=2r`, `target_modules` 覆盖所有 `attention` 和 `MLP` 投影层
支持的参数: `model_name`、`train_file`、`output_dir`、`num_epochs`、`lr`、`bf16/fp16`、`gradient_checkpointing`、`lora_r`、`lora_alpha`、`adapter_name`（从已有 adapter 微调）。

协作关系：输入 = `scripts/` 构建的训练数据 JSONL，输出 = LoRA adapter 权重（保存到 `autodl-tmp/`）。

3.2 train_dpo_lora.py — DPO 偏好训练

解决的问题：在 SFT 基础上通过偏好数据优化答案质量。但由于格式崩溃问题，三次尝试均失败。

核心机制：

- `DPOTrainer` 继承 `Trainer`，重写 `compute_loss`
- 加载 `ref_model`（SFT adapter merge 后冻结）和 `policy_model`（新 LoRA）
- `compute_log_prob`: 只计算最后一条 `assistant` 消息的 `log-prob`
- `DPO loss = -log(sigmoid(beta * (chosen_lp - ref_chosen_lp - rejected_lp + ref_rejected_lp)))`

失败原因: DPO 只优化 last-turn, 不保护多轮格式纪律。详细分析见前文问题 4。

协作关系: 输入 =DPO 偏好数据 JSONL + SFT adapter 路径, 输出 =DPO LoRA 权重。

4 评测系统 (eval/)

4.1 run_vlm_eval.py — VLM 评测主脚本

解决的问题: 加载 VLM 模型 (带 LoRA adapter), 对 50 条评测样本逐一运行 Agent 推理, 记录完整的工具调用 trace 和最终答案, 计算多项指标。

核心指标:

- Tool Hit: gold_tools 是 pred_tools 的子集
- Tool Order Hit: gold_tools 顺序一致
- Format Valid: 无 parse_error
- Finished: 正常结束 (未超 max_steps)
- Answer Keyword Hit: gold_keywords 全部出现在答案中
- Refusal Correct: should_refuse 与 is_refusal 一致

协作关系: 输入 =model_name + adapter_name + eval_path, 输出 =JSONL 评测结果。

4.2 run_seed_eval.py — 早期评测

解决的问题: 项目早期的简化版评测, 不依赖真实 VLM。用于快速验证 Agent 逻辑。

5 数据构建管线 (scripts/)

5.1 核心数据构建

5.1.1 build_first_tool_sft.py

解决的问题: 从完整的 seed 数据 (sft_seed.jsonl) 中提取”用户消息-> 第一个 tool_call” 的训练样本。只保留到第一个工具调用为止, 用于训练模型的第一步决策。

协作关系: 输入 =sft_seed.jsonl -> 输出 =sft_first_tool_seed.jsonl。

5.1.2 build_second_tool_sft.py

解决的问题: 从 seed 数据中提取从开头到第二个工具调用 (retrieve_docs 或 python_exec) 的训练样本。用于增强模型对多工具链的学习。

协作关系: 输入 =sft_seed.jsonl + sft_hard_full_trace.jsonl -> 输出 =sft_second_tool_retrieve_docs.jsonl

5.1.3 build_sft_train.py

解决的问题: 合并 first_tool 和 full_trace 数据, 去重和验证。产出最终的 first_tool_train 和 full_trace_train 数据。

协作关系: 输入 =first_tool_seed + hard_first_tool -> first_tool_train; 输入 =seed + hard_full_trace + hard_format -> full_trace_train。

5.1.4 build_agent_train_v3.py

解决的问题：合并所有 SFT 数据源 (first_tool + full_trace + second_tool) 为一个统一的训练集。

5.1.5 build_agent_train_real_v3.py

解决的问题：合并 SFT 数据 + 外部真实数据集 (ChartQA、DocVQA、CORD)，控制上采样倍率和 recovery 变体数量，产出最终训练集。

核心参数：

- external_sample_ratio: 外部数据采样比例 (0.5=50%)
- python_exec_repeat/retrieve_docs_repeat: 上采样倍率
- recovery_repeat: recovery 变体数量

5.2 外部数据集转换

5.2.1 convert_chartqa_to_sft.py

解决的问题：将 HuggingFace 的 ChartQA 数据集转换为 Agent SFT 格式。每个图表 QA 对包装为 vision_parse(chart_values) -> [python_exec] -> final_answer 的 tool 调用链。

协作关系：通过 -use_real_vlm 标志调用 real_vision_parse.py 进行真实 VLM 推理。

5.2.2 convert_docvqa_to_sft.py

解决的问题：将 DocVQA 数据集转换为 Agent SFT 格式。文档 QA 包装为 vision_parse(ocr) -> [retrieve_docs] -> final_answer。

5.2.3 convert_cord_to_sft.py

解决的问题：将 CORD 收据数据集转换为 Agent SFT 格式。每张收据生成 4 个 QA 对，包装为 vision_parse(ocr) -> retrieve_docs -> final_answer。

5.3 真实 VLM 数据重建

5.3.1 real_vision_parse.py

解决的问题：提供共享的真实 VLM vision_parse 模块。封装 Qwen-VL 模型的加载和视觉推理，供所有转换脚本使用。

5.3.2 rebuild_seed_with_real_vlm.py

解决的问题：用真实 Qwen-VL 模型重建任意 SFT JSONL 文件中的 vision_parse 结果。将 mock/占位文本替换为真实 VLM 输出。支持 image+mode 级缓存。

协作关系：处理 seed 数据 (46 张图)+ChartQA(500 张)+DocVQA(200 张)+CORD(51 张)，共 797 张图片。

5.4 数据增强与修正

5.4.1 build_augmented_v5_data.py

解决的问题：从 seed 数据生成多样化问题的增强训练数据。包括：多样问法模板（每个样本 ×5-7 种问法）、推理链前缀、纠错轨迹。

5.4.2 build_hard_cases_from_eval.py

解决的问题：从评测错误中自动生成 SFT 修正样本和 DPO 偏好对。错误类型包括：missing_tool_call、wrong_tool_selection、over_refusal、should_refuse_but_answered 等。

协作关系：输入 =tag_vlm_errors.py 的输出（tagged eval 结果），输出 =hard_first_tool + hard_format + hard_full_trace + dpo_hard_cases。

5.4.3 build_dpo_multiturn_v4.py

解决的问题：构建多轮 DPO 偏好对。与单轮 DPO 不同，prompt 包含完整的工具调用上下文，只优化最后一步决策。

5.4.4 build_refusal_samples.py

解决的问题：用真实 VLM（7B）对 logo 图片做 vision_parse，将“数字/乱码->拒答”的正确流程生成为 SFT 训练样本。

5.4.5 build_tool_decision_samples.py

解决的问题：生成教模型“何时不调工具”和“何时调工具后拒答”的训练样本。包括带推理前缀的直接回答样本和工具调用 + 拒答样本。

5.5 诊断与分析

5.5.1 tag_vlm_errors.py

解决的问题：对评测结果进行自动错误标注。将每个样本分类为 ok、missing_tool_call、wrong_tool_selection、answer_keyword_miss 等类别。

5.5.2 analyze_sft_train.py, analyze_vlm_eval.py, analyze_error_tags.py

解决的问题：训练数据和评测结果的分析工具。统计工具调用分布、类别分布、错误标签分布。

5.5.3 add_eval_category.py, check_seed_data.py

解决的问题：eval 数据集预处理（添加 category 标签）和 seed 数据完整性检查。

6 训练数据 (data/)

6.1 SFT 训练数据

文件	说明
sft_seed.jsonl	50 条种子数据。手动构造的 mock vision_parse 结果的完整 tool 调用链
sft_first_tool_train.jsonl	84 条。用户消息-> 第一个 tool_call (训练第一步决策)
sft_full_trace_train.jsonl	85 条。完整的多轮工具调用轨迹
sft_hard_first_tool.jsonl	34 条。从评测错误中生成的 first_tool 修正样本
sft_hard_full_trace.jsonl	34 条。从评测错误中生成的 full_trace 修正样本
sft_hard_format.jsonl	1 条。格式修正样本
sft_agent_train_real_v4_enforced_resized.jsonl	1133 条。V4/V7B 训练数据 (核心)。合并 seed+hard+外部数据集, resize 图片版本
sft_agent_train_v7b_final.jsonl	1136 条。V7B-final 训练数据 (V4 数据 +3 条 refusal 样本)
sft_refusal_boundary_v7.jsonl	9 条。用 7B 真实 VLM 生成的防幻觉样本
sft_tool_decision_v8.jsonl	53 条。工具决策训练样本 (直接回答 +refusal+tool)

6.2 评测数据

文件	说明
eval_dev.jsonl	50 条。评测集 (不含 category 标签)
eval_dev_with_category.jsonl	50 条。带 category 标签的评测集 (使用中)
eval_test.jsonl	空文件 (测试集未构建)

6.3 DPO 偏好数据

文件	说明
dpo_seed.jsonl	30 条。种子 DPO 偏好对
dpo_hard_cases.jsonl	28 条。从 V3 评测错误生成的 DPO 硬 case
dpo_train_v4_mt.jsonl	13 条。多轮 DPO 偏好对 (V4 错误修正)
dpo_visionarena.jsonl	空文件 (未使用)

6.4 规则文档 (data/docs/)

文件	说明
invoice_rules.txt	发票必填字段规则: Invoice No, Date, Amount, Tax ID

chart_rules.txt	图表计算规则: 同比增长率 = (当前值-基期值)/基期值 ×100%
product_rules.txt	商品图文一致性规则: 颜色、品类、品牌任意冲突则不一致

6.5 图片资源 (data/images/)

解决的问题: 提供训练和评测用的视觉输入。

类别	文件
图表 (12 张)	chart_001.png – chart_012.png
发票 (17 张)	invoice_001.jpg – invoice_017.jpg
品牌 Logo (7 张)	logo_001.jpg – logo_007.jpg
商品图片 (17 张)	product_001.jpg – product_017.jpg

共 53 张图片, 覆盖 4 个任务类型。Seed 数据引用其中 46 张。

7 模型权重 (autodl-tmp/multimodal-agent-lora/)

模型	LoRA 配置	状态
lora_agent_real_v2	3B, r=16, mock 数据, 446 条	基准线
lora_agent_real_v4	3B, r=16, 真实 VLM 数据, 1133 条	3B 最优
lora_agent_v7b_clean	7B, r=32, 真实 VLM 数据, 1133 条	7B 最优

8 外部图片资源 (autodl-tmp/multimodal-agent-data/)

目录	内容	说明
chartqa/images_resized/	500 张	ChartQA 图表, 已 resize 到 1024px
docvqa/images_resized/	200 张	DocVQA 文档, 已 resize
cord/images_resized/	51 张	CORD 收据, 已 resize (原图 203 条, 51 张去重)

原始大图已删除 (OOM 问题, 仅保留 resize 版本)。

9 完整 workflow

9.1 训练数据构建流程

1. sft_seed.jsonl (50条, 手动构造)

2. build_first_tool_sft.py -> sft_first_tool_seed.jsonl
3. build_second_tool_sft.py -> sft_second_tool_retrieve_docs.jsonl
4. build_sft_train.py -> sft_first_tool_train.jsonl + sft_full_trace_train.jsonl
5. convert_chartqa_to_sft.py --use_real_vlm -> sft_chartqa_real_vlm.jsonl
6. convert_docvqa_to_sft.py --use_real_vlm -> sft_docvqa_real_vlm.jsonl
7. convert_cord_to_sft.py --use_real_vlm -> sft_cord_real_vlm.jsonl
8. rebuild_seed_with_real_vlm.py -> sft_seed_real_vlm.jsonl
9. build_agent_train_real_v3.py -> sft_agent_train_real_v4_enforced.jsonl
10. resize外部图片 -> *_resized/ 目录
11. 更新JSONL路径 -> sft_agent_train_real_v4_enforced_resized.jsonl

9.2 训练流程

train_sft_lora.py

```
--model_name /path/to/Qwen-VL
--train_file sft_agent_train_real_v4_enforced_resized.jsonl
--output_dir /path/to/lora_adapter
--bf16 --lora_r 16 --gradient_accumulation_steps 8
-> LoRA adapter权重
```

9.3 评测流程

run_vlm_eval.py

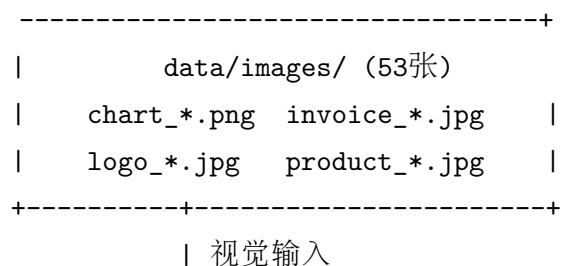
```
--model_name /path/to/Qwen-VL
--adapter_name /path/to/lora_adapter
--enforce_required_tools True/False
-> JSONL评测结果 (trace + metrics)
```

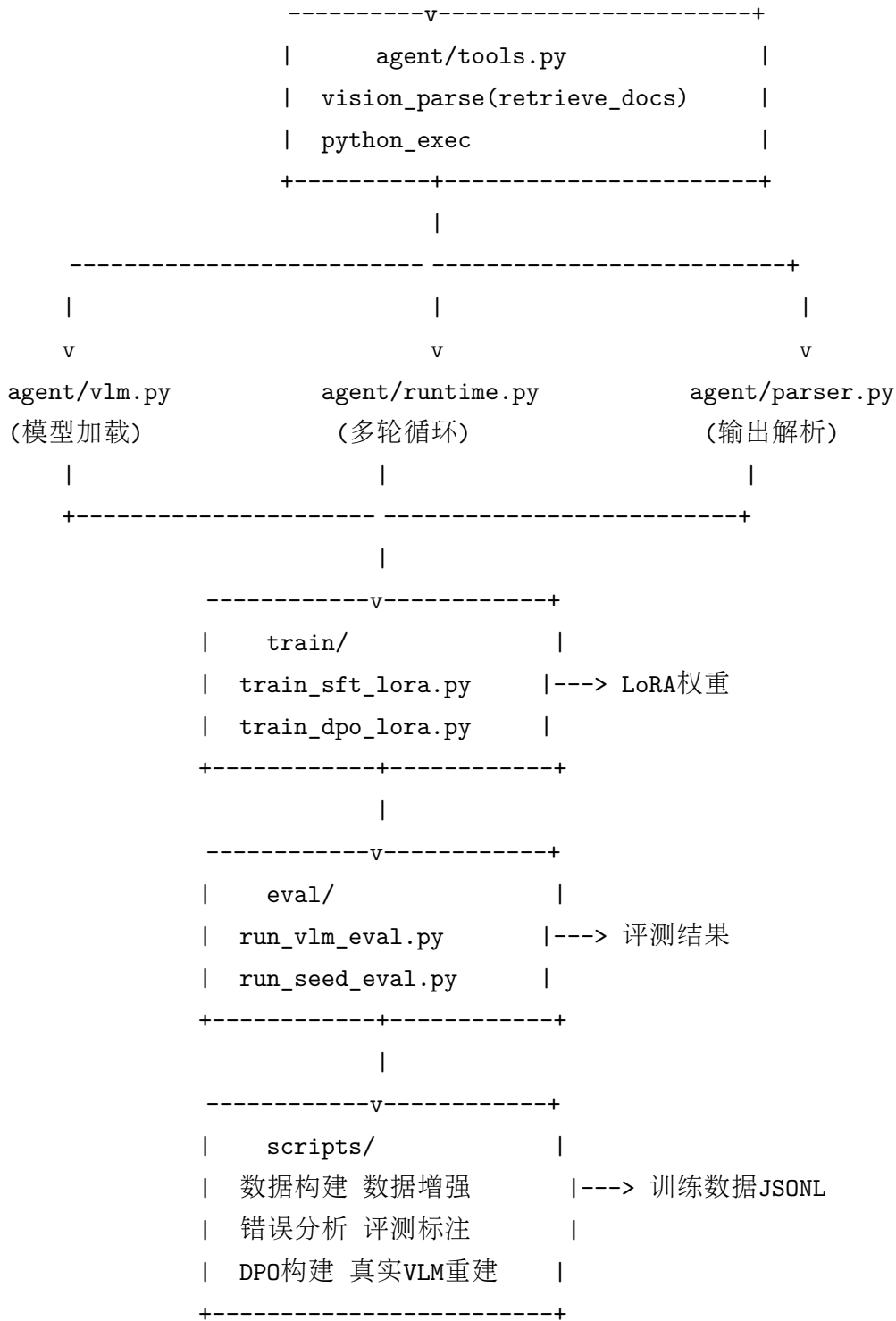
9.4 推理流程

QwenVLModel(model_name, adapter_name).generate(messages)

```
-> runtime.run(image, question)
-> 多轮循环: VLM生成 -> parser解析 -> 工具执行 -> 结果反馈
-> {answer, trace, status}
```

10 文件协作关系图





11 总结

本项目共包含约 **90** 个文件，按功能分为 **6** 个模块：

1. **Agent** 推理核心 (6 个 py): runtime.py、tools.py、vlm.py、parser.py、prompts.py、rule_agent.py。负责多轮 Agent 执行、工具调用、输出解析。
2. 训练引擎 (2 个 py): train_sft_lora.py、train_dpo_lora.py。负责 SFT 和 DPO 的 LoRA 训练。
3. 评测系统 (2 个 py): run_vlm_eval.py、run_seed_eval.py。负责模型评测和指标计算。

4. 数据构建管线 (25 个 py): 覆盖外部数据集转换、真实 VLM 数据重建、数据增强、错误分析、偏好对构建、工具决策样本生成等全流程。
5. 训练数据 (35 个 jsonl/txt): SFT 数据、DPO 偏好对、评测数据、规则文档。
6. 模型与图片 (850 个文件): 3 个 LoRA adapter、2 个基座模型 (3B+7B)、53 张本地图片 +751 张外部 resize 图片。

所有模块围绕一个核心目标协作: 从数据构建->**SFT** 训练-> 评测反馈-> 数据修正, 形成闭环迭代。关键瓶颈不在于某个模块, 而在于 SFT 优化范式的根本局限—这一发现已记录于前文分析中, 后续 RL/GRPO 将是突破路径。